



Real-Time Big Data Pipeline for NASA/JPL Planetary Defense Tracking

CDAC | DBDA Course Project | Big Data Engineering & Stream Processing

Author	Date	Status	Version
Talentum	June 12, 2026	✓ In Progress	v1.0

i This document covers the complete architecture, implementation, data schema, code walkthrough, results, and next steps for the NASA asteroid real-time big data pipeline built on BigDataVM.

1. Project Overview

NASA tracks over 1.3 million asteroids, comets, and Near-Earth Objects (NEOs) that come close to Earth. This project builds an end-to-end real-time big data pipeline that ingests live asteroid telemetry, processes it at scale, stores it efficiently, and visualizes it on an interactive dashboard — functioning as a mini planetary defense early-warning data system.

✓ The core pipeline ingests live NASA asteroid data every 60 seconds, processes it through Kafka and Spark Streaming, stores it as Parquet on HDFS, queries it via Hive SQL, and visualizes it in Tableau — all running on a single Ubuntu VM.

1.1 Objectives

- ▶ Build a fault-tolerant, real-time data ingestion pipeline using Apache Kafka
- ▶ Process live NASA asteroid telemetry using PySpark Structured Streaming
- ▶ Store data efficiently using Parquet columnar format on HDFS (~70% storage reduction)
- ▶ Enable SQL analytics via Apache Hive external tables over the data lake
- ▶ Visualize asteroid hazard scores, velocities, and close-approach trends in Tableau
- ▶ Demonstrate production-grade patterns: schema enforcement, fault tolerance, deduplication

1.2 Scope

In Scope	Out of Scope
Live NASA/JPL NeoWs API integration	Historical asteroid data (pre-2026)
Real-time Kafka → Spark → HDFS pipeline	Multi-node Hadoop cluster deployment
Hive SQL analytics and aggregations	Machine learning / impact prediction
Tableau dashboard (3 charts + dashboard)	Real-time alerts / notification system
Data deduplication and quality checks	Cloud deployment (AWS/GCP/Azure)

2. System Architecture

The pipeline consists of six layers connected in a linear data flow:

NASA/JPL API → Kafka Producer → Kafka Broker → Spark Streaming → Parquet/HDFS → Hive SQL → Tableau

#	Layer	Technology	Responsibility	Port/Path
1	Ingest	NASA/JPL REST API	Polls live asteroid close-approach data every 60 seconds	api.nasa.gov
2	Queue	Apache Kafka 2.x	Fault-tolerant message buffer, 3 partitions, offset tracking	localhost:9092
3	Process	PySpark 2.4.5	Structured Streaming, schema enforcement, micro-batches every 15s	local[*]
4	Store	Parquet + HDFS 2.7	Columnar Snappy-compressed storage, partitioned on HDFS	localhost:9000
5	Query	Apache Hive 2.3.6	External table over Parquet lake, SQL analytics, GROUP BY	localhost:10000
6	Visualize	Tableau Public	3-chart interactive dashboard: hazard, velocity, miss distance	Windows

2.1 Environment

Component	Details
VM Name	BigDataVM (Oracle VirtualBox)
OS	Ubuntu 64-bit
RAM	10 GB
CPUs	9 vCPUs
Disk	150 GB (SCSI)
Username	talentum

3. Data Schema

Each asteroid record flowing through the pipeline is a flat JSON object with 11 fields. The producer flattens NASA's deeply nested JSON into this simple structure for easy Spark schema enforcement.

Field	Type	Source	Description
id	STRING	NASA API	Unique SPK-ID assigned by NASA JPL
name	STRING	NASA API	IAU designation e.g. '388945 (2008 TZ3)'
hazardous	BOOLEAN	NASA API	true if classified as Potentially Hazardous Asteroid (PHA)
magnitude	DOUBLE	NASA API	Absolute magnitude H — proxy for asteroid size/brightness
diameter_km_min	DOUBLE	NASA API	Estimated minimum diameter in kilometers
diameter_km_max	DOUBLE	NASA API	Estimated maximum diameter in kilometers
velocity_km_s	STRING	NASA API	Relative velocity in km/s (STRING to preserve full precision)
miss_dist_km	STRING	NASA API	Miss distance from Earth center in km (STRING to preserve precision)
close_approach	STRING	NASA API	Date of closest approach (YYYY-MM-DD)
orbiting_body	STRING	NASA API	Body being orbited — almost always 'Earth'
fetches_at	STRING	Producer	UTC ISO timestamp when record was fetched from NASA API

⚠ velocity_km_s and miss_dist_km are stored as STRING intentionally. Converting to DoubleType introduces IEEE 754 floating-point rounding errors which corrupt orbital mechanics calculations. We cast to DOUBLE only at query time: CAST(velocity_km_s AS DOUBLE).

4. Code Walkthrough

4.1 NASA Producer (nasa_producer.py)

The producer polls the NASA NeoWs API every 60 seconds and pushes flattened asteroid JSON to Kafka.

Key functions:

- ▶ `fetch_asteroids()` — calls the NASA API with today's date range, returns raw JSON
- ▶ `flatten_asteroid()` — converts NASA's 4-level nested JSON into a flat 11-field dict
- ▶ Main loop — sends each record to Kafka topic 'asteroids', flushes every 60 seconds

 Why flatten? NASA's raw JSON has `close_approach_data` as a list inside a dict inside a dict. Spark can parse nested JSON but it's complex. Flattening at the producer keeps the Spark schema simple and readable — a standard production pattern called schema normalization at the source.

API URL: `https://api.nasa.gov/neo/rest/v1/feed?start_date=YYYY-MM-DD&end_date=YYYY-MM-DD&api_key=KEY`

Kafka send: `producer.send('asteroids', value=flat_dict)`

Poll interval: 60 seconds

4.2 Spark Streaming Job (nasa_spark.py)

The Spark job reads from the Kafka 'asteroids' topic, enforces schema, and writes Parquet files to HDFS every 15 seconds.

Processing steps:

1. `readStream` from Kafka — reads raw bytes from 'asteroids' topic, starting from earliest offset
2. Cast value to `STRING` — Kafka messages arrive as binary; cast to UTF-8 string first
3. `from_json()` with `StructType` schema — parse JSON with strict typing (prevents corrupt data)
4. `writeStream` to HDFS — `format('parquet')` with `checkpointLocation` for fault tolerance

 Important: Always run `'unset PYSARK_DRIVER_PYTHON && unset PYSARK_DRIVER_PYTHON_OPTS'` before `spark-submit` on this VM. The environment has `PYSARK_DRIVER_PYTHON` set to Jupyter by default which causes `spark-submit` to fail with a Jupyter error.

Command: `spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.4.5 nasa_spark.py`

Output: `hdfs://localhost:9000/user/talentum/asteroids/*.snappy.parquet`

Checkpoint: `hdfs://localhost:9000/user/talentum/asteroids_checkpoint/`

4.3 Hive External Table (create_table.sql)

An EXTERNAL Hive table is mapped directly over the Parquet files on HDFS. No data movement or copying required.

```
CREATE EXTERNAL TABLE asteroids (...) STORED AS PARQUET LOCATION  
'hdfs://localhost:9000/user/talentum/asteroids/';
```

✓ **EXTERNAL vs MANAGED:** An EXTERNAL table means Hive only holds metadata — the Parquet files stay on HDFS under Spark's ownership. DROP TABLE deletes only the metadata, not the data files. This is the correct pattern when Spark owns the data and Hive only reads it.

5. Setup & Startup Procedure

⚠ Always start services in this exact order. Starting Kafka before Zookeeper, or Spark before HDFS, will cause connection errors.

S t e p	Service	Command	Verify
1	HDFS	~/hadoop/sbin/start-dfs.sh	hdfs dfs -ls /
2	YARN	~/hadoop/sbin/start-yarn.sh	jps → ResourceManager
3	Zookeeper	~/kafka/bin/zookeeper-server-start.sh ~/kafka/config/zookeeper.properties	Port 2181 listening
4	Kafka Broker	~/kafka/bin/kafka-server-start.sh ~/kafka/config/server.properties	jps → Kafka
5	Hive Metastore	~/run-hivemetastore.sh -s start &	Port 9083 listening
6	HiveServer2	hive --service hiveserver2 &	netstat grep 10000

5.1 Create Kafka Topic

```
~/kafka/bin/kafka-topics.sh --create --topic asteroids --bootstrap-server localhost:9092
--partitions 3 --replication-factor 1
```

5.2 Run the Full Pipeline

Terminal	Command
Terminal 1	~/kafka/bin/zookeeper-server-start.sh ~/kafka/config/zookeeper.properties
Terminal 2	~/kafka/bin/kafka-server-start.sh ~/kafka/config/server.properties
Terminal 3	python3 ~/nasa_producer.py
Terminal 4	unset PYSPARK_DRIVER_PYTHON && unset PYSPARK_DRIVER_PYTHON_OPTS && spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.11:2.4.5 ~/nasa_spark.py
Terminal 5	hive -f hive/create_table.sql (first time only)

6. Results & Live Data Findings

The following results were obtained from the pipeline running live on June 12, 2026:

207+ Asteroids Ingested	2 Hazardous Asteroids	~70% Storage Reduction	15s Micro-Batch Interval
-----------------------------------	---------------------------------	----------------------------------	------------------------------------

6.1 Notable Asteroids Detected

Asteroid	Hazardous	Miss Distance (km)	Velocity (km/s)	Date
388945 (2008 TZ3)	⚠ YES	16,794,474	7.48	2026-06-12
530520 (2011 LT17)	⚠ YES	6,155,491	14.57	2026-06-12
(2002 UQ12)	No	70,108,809	35.32 🏆 fastest	2026-06-12
(2016 VS)	No	7,645,400 📍 closest	11.08	2026-06-12

 530520 (2011 LT17) passed at only 6,155,491 km — approximately 16× the Earth-Moon distance. While not a threat today, it is classified as potentially hazardous due to its size (~250m diameter) and orbital proximity.

6.2 Hive SQL Query Results

Query 1: Total count

```
SELECT COUNT(*) FROM asteroids; → Result: 207
```

Query 2: Hazardous breakdown

```
SELECT hazardous, COUNT(*) FROM asteroids GROUP BY hazardous; → false: 184 | true: 23
```

Query 3: Top 5 fastest

```
SELECT name, velocity_km_s FROM asteroids ORDER BY CAST(velocity_km_s AS DOUBLE) DESC LIMIT 5;
→ (2002 UQ12): 35.32 km/s | 530520 (2011 LT17): 14.57 km/s | ...
```


7. Known Issues & Fixes

Issue	Root Cause	Fix
Duplicate records in CSV/Hive	Producer polls every 60s, same asteroids re-sent	Add dropDuplicates(['id','close_approach']) in Spark, or SELECT DISTINCT in Hive
spark-submit opens Jupyter instead of running script	PYSPARK_DRIVER_PYTHON env var set to Jupyter	Run: unset PYSPARK_DRIVER_PYTHON && unset PYSPARK_DRIVER_PYTHON_OPTS before every spark-submit
Hive metastore connection refused	Hive metastore not started, or MySQL not running	sudo service mysql start && ~/run-hivemetastore.sh -s start &
Start-Hadoop-Hive.sh fails with file not found	run-hdfs.sh and run-yarn.sh missing from home dir	Start manually: ~/hadoop/sbin/start-dfs.sh && ~/hadoop/sbin/start-yarn.sh
CSV file busy when writing from VM	Tableau on Windows has a file lock on the CSV	Write to a temp file first (asteroids_final.csv) then copy to shared folder with a different name

8. Tableau Dashboard

The dashboard visualizes live asteroid data exported from Hive as CSV and loaded into Tableau Public on Windows.

#	Chart	Configuration	Insight
1	Hazardous vs Non-Hazardous	Columns: Hazardous Rows: COUNT(Id) Type: Bar	~280 safe vs ~80 hazardous asteroids
2	Velocity Scatter Plot	Columns: Name Rows: Velocity Km S Color: Hazardous Size: Velocity	Hazardous (orange) vs safe (blue) clearly separated
3	Miss Distance Chart	Rows: Name Columns: Miss Dist Km Color: Hazardous Sort: DESC	Closest asteroids highlighted in orange

9. Pending Work

#	Task	Description	Priority
---	------	-------------	----------

1	Fix duplicate data	Add dropDuplicates() in Spark and SELECT DISTINCT in Hive export	 High
2	Tableau Chart 3 + Dashboard	Build miss distance chart and combine all 3 into a single dashboard view	 Medium
3	GitHub repository	Extract zip, add all project files, commit and push to GitHub	 Medium
4	Hive GROUP BY CUBE queries	Pre-aggregate hazard counts by orbital class and approach date	 Low
5	Final project report	Full documentation with screenshots and architecture diagram	 Low

10. References

- ▶ NASA NeoWs API Documentation: <https://api.nasa.gov>
- ▶ Apache Kafka Documentation: <https://kafka.apache.org/documentation/>
- ▶ PySpark Structured Streaming Guide: <https://spark.apache.org/docs/2.4.5/structured-streaming-kafka-integration.html>
- ▶ Apache Hive Language Manual: <https://cwiki.apache.org/confluence/display/Hive/LanguageManual>
- ▶ Apache Parquet Format: <https://parquet.apache.org/docs/>
- ▶ Tableau Public: <https://public.tableau.com>

This document was prepared as part of the CDAC DBDA Program, June 2026 | Talentum